In [1]:
```
# ---------- DFS using Adjacency Matrix ----------
class GraphMatrix:
    def __init__(self, vertices):
        self.vertices = vertices
        self.size = len(vertices)
        self.adj_matrix = [[0] * self.size for _ in range(self.size)]

    def add_edge(self, u, v):
        if u not in self.vertices or v not in self.vertices:
            print(f"Error: One or both vertices '{u}', '{v}' not found
            return
        i, j = self.vertices.index(u), self.vertices.index(v)
        self.adj_matrix[i][j] = 1
        self.adj_matrix[j][i] = 1  # Undirected graph

    def dfs(self, start, visited=None):
        if start not in self.vertices:
            print(f"Error: Starting vertex '{start}' not found.")
            return

        if visited is None:
            visited = set()
        visited.add(start)
        print(start, end=" ")

        start_index = self.vertices.index(start)
        for i in range(self.size):
            if self.adj_matrix[start_index][i] == 1 and self.vertices[i
                self.dfs(self.vertices[i], visited)


# ---------- BFS using Adjacency List ----------
class GraphList:
    def __init__(self):
        self.adj_list = {}

    def add_edge(self, u, v):
        # Ensure both vertices are present
        if u not in self.adj_list:
            self.adj_list[u] = []
        if v not in self.adj_list:
            self.adj_list[v] = []
        self.adj_list[u].append(v)
        self.adj_list[v].append(u)  # Undirected graph

    def bfs(self, start):
        if start not in self.adj_list:
            print(f"Error: Starting vertex '{start}' not found in adjac
            return

        visited = set()
        queue = [start]
        visited.add(start)
```

```
        while queue:
            node = queue.pop(0)  # FIFO
            print(node, end=" ")

            for neighbor in self.adj_list[node]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)


# ---------------- Main Program ----------------
if __name__ == "__main__":
    print("Graph Traversal - City Area Example")
    n = int(input("Enter number of locations: "))

    # Take location names as input
    vertices = []
    for i in range(n):
        loc = input(f"Enter location {i+1} name (e.g., A, B, C): ").upp
        vertices.append(loc)

    # Create graphs
    g_matrix = GraphMatrix(vertices)
    g_list = GraphList()

    # Enter routes between locations
    e = int(input("Enter number of routes (edges): "))
    print("Enter routes (e.g., A B means there is a route between A and
    for _ in range(e):
        u, v = input().split()
        u, v = u.upper(), v.upper()
        g_matrix.add_edge(u, v)
        g_list.add_edge(u, v)

    # Starting location
    start = input("Enter starting location: ").upper()

    print("\nDFS Traversal using Adjacency Matrix:")
    g_matrix.dfs(start)

    print("\n\nBFS Traversal using Adjacency List:")
    g_list.bfs(start)

    print()  # for clean newline at the end
```

```
Graph Traversal - City Area Example
Enter number of locations: 2
Enter location 1 name (e.g., A, B, C): A
Enter location 2 name (e.g., A, B, C): B
Enter number of routes (edges): 1
Enter routes (e.g., A B means there is a route between A and B):
A B
Enter starting location: A

DFS Traversal using Adjacency Matrix:
A B

BFS Traversal using Adjacency List:
A B
```

In [ ]: